

BÁO CÁO BÀI TẬP LỚN

Đề tài: Hệ thống đấu giá trực tuyến - Nhóm 3

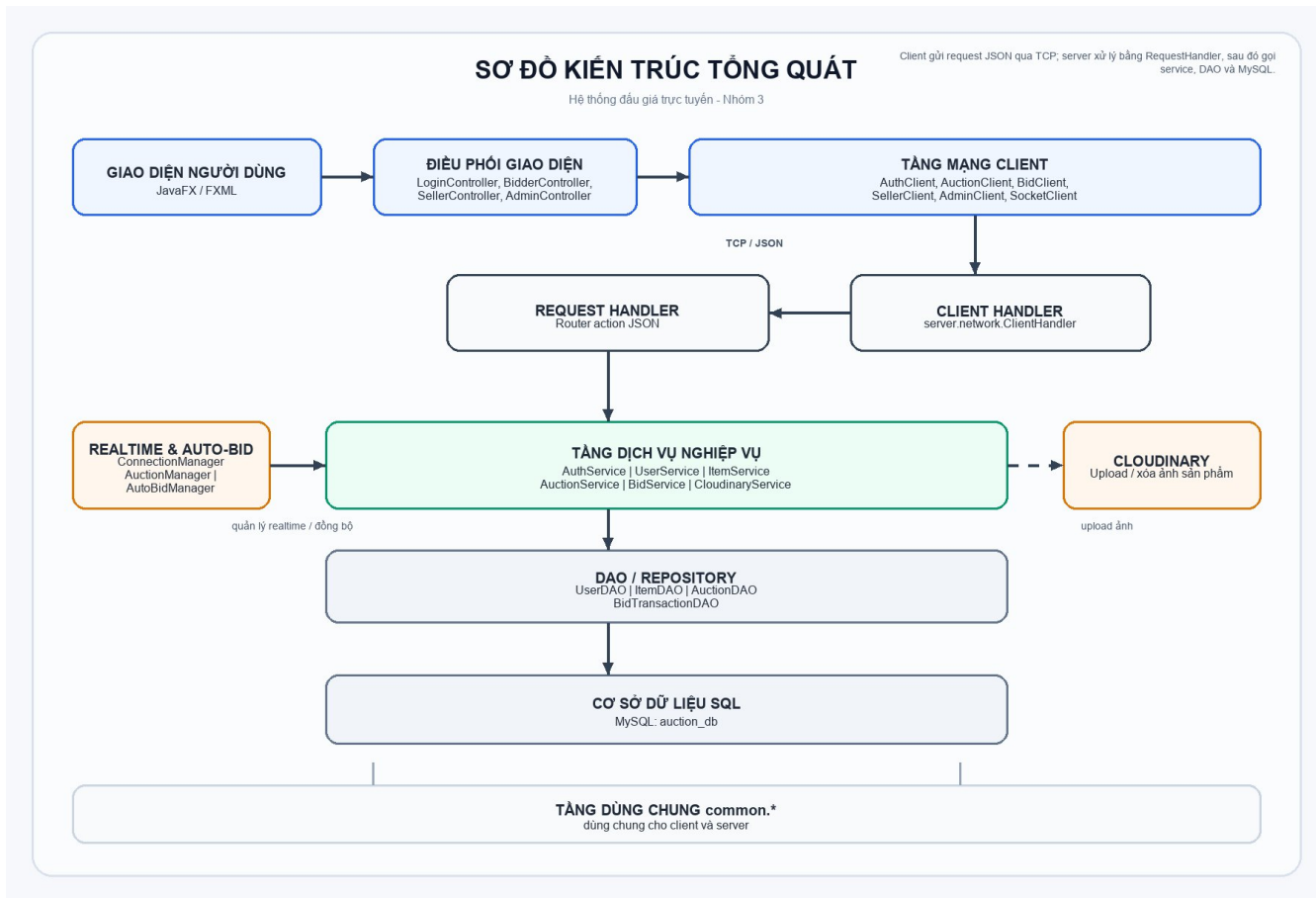
Nguồn lập báo cáo: mã nguồn hiện tại, file đề bài *2026-Bài tập lớn.pdf* và lịch sử commit của project.

1. Mục tiêu và phạm vi thực hiện

Mục tiêu của project là xây dựng ứng dụng đấu giá trực tuyến theo mô hình hướng đối tượng, có giao diện desktop JavaFX, giao tiếp client-server qua socket JSON và chỉ phía server được truy cập cơ sở dữ liệu. Hệ thống phục vụ ba nhóm người dùng chính: Bidder đặt giá, Seller đăng/sửa/kết thúc phiên đấu giá, Admin quản trị người dùng và phiên đấu giá.

Phạm vi đã triển khai gồm: đăng ký/đăng nhập bằng mật khẩu mã hóa và JWT, refresh token, quản lý tài khoản và ví, quản lý sản phẩm/ảnh sản phẩm, tạo/cập nhật/kết thúc/hủy phiên đấu giá, đặt giá thủ công, đấu giá tự động, chống sniping, lịch sử và biểu đồ giá, cập nhật realtime qua socket, kiểm thử JUnit, JaCoCo, Checkstyle và CI/CD bằng GitHub Actions.

2. Sơ đồ kiến trúc tổng quát



Hình 1. Sơ đồ kiến trúc tổng quát của hệ thống.

Mô tả các thành phần kiến trúc:

Tầng Client: Giao diện JavaFX/FXML cùng các controller (LoginController, BidderController, SellerController, AdminController, AuctionDetailController) chịu trách nhiệm hiển thị, xử lý thao tác người dùng và gọi các client mạng chuyên trách cho từng nghiệp vụ.

Giao thức truyền thông: SocketClient gửi và nhận dữ liệu dạng JSON trên TCP, hỗ trợ request/response đồng bộ, push realtime từ server và cơ chế tự refresh access token khi hết hạn.

Tầng Server: ClientHandler tiếp nhận từng kết nối, RequestHandler định tuyến theo action; các service AuthService, UserService, ItemService, AuctionService, BidService và CloudinaryService xử lý nghiệp vụ, trong khi AuctionServer dùng thread pool và scheduler để tiếp nhận nhiều client đồng thời và cập nhật trạng thái phiên đấu giá định kỳ.

Tầng lưu trữ và tích hợp: DAO/Repository ghi đọc dữ liệu từ MySQL auction_db; ảnh sản phẩm được tải lên Cloudinary, còn các lớp trong common.* được dùng chung cho cả client và server để đảm bảo đồng bộ mô hình dữ liệu.

3. Chức năng đạt được theo barem điểm

Barem	Chức năng đạt được	Hướng giải quyết	Lý do lựa chọn
Thiết kế lớp, kế thừa	Đủ Entity, User, Bidder/Seller/Admin, Item, Art/Electronics/Vehicle, Auction, Bid-Transaction.	Dùng abstract class Entity, User, Item; subtype kế thừa và override getInfo().	Đúng yêu cầu OOP, dễ mở rộng thêm role hoặc loại sản phẩm.
Nguyên tắc OOP	Đóng gói dữ liệu, kế thừa, đa hình, trừu tượng hóa.	Thuộc tính private/protected, getter/setter, service validate trước khi thao tác.	Giảm phụ thuộc trực tiếp và bảo vệ trạng thái nghiệp vụ.
Design pattern	Singleton, Factory Method, Observer/Socket push, Repository/DAO, MVC.	AuctionManager/ConnectionManager/AutoBidManager dùng Singleton; UserFactory/ItemFactory tạo subtype; ConnectionManager broadcast sự kiện.	Phù hợp bài toán nhiều client, nhiều loại entity và cần realtime.
Quản lý người dùng, sản phẩm	Register/login/logout, ban/unban, switch role, ví tiền, tạo/sửa sản phẩm và ảnh.	UserService/AuthService/ItemService; mật khẩu hash; JWT/refresh token; CloudinaryService upload ảnh.	Tách xác thực, hồ sơ, sản phẩm và ảnh để dễ kiểm thử, tránh client chạm DB.
Chức năng đấu giá	Tạo, cập nhật khi OPEN, kết thúc, hủy, xem danh sách/chi tiết, đặt giá, lịch sử bid.	AuctionService quản lý vòng đời; BidService xử lý đặt giá; DAO lưu Auction và BidTransaction.	Đảm bảo trạng thái phiên nhất quán và truy vết được toàn bộ giao dịch.
Xử lý lỗi & ngoại lệ	Chặn dữ liệu thiếu/sai, tài khoản bị khóa, seller tự đấu giá, thiếu số dư, phiên đóng.	AuthenticationException, InvalidBidException, AuctionClosedException; RequestHandler gom lỗi thành JSON error.	Client nhận lỗi thống nhất, server không rò rỉ lỗi nội bộ.
Concurrency an toàn	Tránh lost update/race khi nhiều bidder đặt giá cùng lúc.	ReentrantLock theo bidder và auction, transaction DB, rollback và restore trạng thái Auction khi persist lỗi.	Khóa đúng phạm vi phiên đấu giá, vẫn cho các phiên khác xử lý song song.
Realtime update	Bid mới, phiên tạo/sửa/kết thúc/hủy được đẩy tới client đang mở.	ConnectionManager broadcast; SocketClient listener dispatch push; controller refresh UI và chart.	Không cần polling liên tục, giảm tải và cập nhật nhanh hơn.

Barem	Chức năng đạt được	Hướng giải quyết	Lý do lựa chọn
Client-Server	Client không truy cập database; giao tiếp qua socket JSON.	AuctionServer + ClientHandler + RequestHandler; network client phía JavaFX chỉ gửi action/payload.	Đúng yêu cầu phân tầng, bảo mật và dễ triển khai nhiều client.
MVC	Client JavaFX + FXML; server Controller-Service-DAO/Model.	FXML views tách khỏi controller; RequestHandler như controller server; service xử lý nghiệp vụ.	Dễ bảo trì giao diện và nghiệp vụ độc lập.
Maven, convention, clean code	Maven wrapper, JavaFX, Gson, MySQL, JWT, Cloudinary, JUnit, JaCoCo, Checkstyle.	pom.xml quản lý dependency/plugin; checkstyle.xml kiểm tra quy ước.	Build lặp lại được, thuận tiện CI và chấm bài.
Unit Test	Có test cho service, manager, DAO, network, controller logic/UI light, model/factory.	JUnit 5 + Surefire + JaCoCo; hiện có 35 test suite, 329 test, 0 failure/error trong báo cáo Surefire.	Bảo vệ các logic rủi ro cao: auth, bid, auto-bid, lock, network.
CI/CD	CI chạy build/test tự động; CD build release jar.	.github/workflows/ci.yml chạy JDK 25, Xvfb và ./mvnw clean test; cd.yml build package và tạo release.	Phát hiện lỗi sớm, phù hợp project JavaFX có test headless.
Nâng cao	Auto-Bidding, Anti-sniping, realtime price chart, ảnh sản phẩm, refresh token.	AutoBidManager dùng PriorityQueue và executor tuần tự; Auction.checkAndExtendForSniping gia hạn; LineChart cập nhật theo bid history.	Tăng tính thực tế và bám sát mục tùy chọn của barem.

4. Phân chia công việc trong nhóm

Thành viên	Mảng phụ trách chính	File/commit tiêu biểu	Kết quả
Nghĩa	Kiểm thử diện rộng; sửa lỗi bid/concurrency/rollback; hoàn thiện auto-bid, ví, timer và màn chi tiết.	BidService.java, AuctionService.java, AutoBidManager.java, AuctionLockManager.java, AuctionDetailController.java; nhiều test trong server.service, server.manager, client.controller.	Tăng độ ổn định logic trọng yếu; 329 test hiện pass trong Surefire.
Lê Duy Mạnh	Luồng Seller và sản phẩm; upload ảnh; JWT/refresh token; role switching; các fix đăng nhập/admin.	SellerController.java, SellerClient.java, CloudinaryService.java, JwtUtil.java, AuthService.java, RequestHandler.java; commits về refresh token, mô tả/ảnh sản phẩm, khóa ban admin.	Seller tạo/sửa phiên đầy đủ hơn; xác thực bảo mật hơn và ảnh sản phẩm hiển thị được.
Phúc	Hạ tầng client-server/socket; realtime push; refactor client không gọi service trực tiếp; checkstyle.	SocketClient.java, ConnectionManager.java, ClientHandler.java, RequestHandler.java, AuctionServer.java, checkstyle.xml; commits chuyển polling sang push realtime.	Kiến trúc mạng rõ ràng, server quản lý DB tập trung, realtime ổn định hơn.
Anh Việt	Dựng nền model/repository, UI/FXML ban đầu và chỉnh các màn Bid-	AdminDashboardView.fxml, BidderDashboardView.fxml, SellerDashboardView.fxml, AuctionDetailsView.fxml, các model Auction/BidTransac-	Tạo nền giao diện và cấu trúc dữ liệu để các chức năng

Thành viên	Mảng phụ trách chính	File/commit tiêu biểu	Kết quả
	der/Admin/Detail.	tion/User/Item và DAO/repository.	sau tích hợp lên.

5. Kết luận

Project đã hoàn thành các yêu cầu bắt buộc của barem: thiết kế OOP, kiến trúc Client-Server, MVC, quản lý người dùng/sản phẩm, đấu giá, xử lý lỗi, concurrency an toàn, realtime update, Maven, test và CI/CD. Ngoài ra nhóm đã triển khai nhiều chức năng nâng cao như Auto-Bidding, Anti-sniping, biểu đồ giá real-time, ảnh sản phẩm và refresh token. Các phần còn có thể cải thiện thêm là chuẩn hóa UI, bổ sung tài liệu hướng dẫn chạy và mở rộng kiểm thử tích hợp với database thật.